GATE
CS & DA

Artificial Intelligence

Informed Search

Lecture No. 7

By- SIDDHARTH SABHARWAL SIR

Recap of Previous Lecture

- Topic
- Topic
- Topic
- Topic
- Topic

IDA*

Topics to be Covered

Topic IDA*

Topic BFHS

Topic Hill climbing Search

Topic

Topic

IDA* ⇒ optimal, TC = O(b^d)
SC = O(b×d)

Iterative Deepening A* algorithm (IDA*) – Artificial intelligence

- The Iterative Deepening A* (IDA*) algorithm is an extension of the A* search algorithm designed to use less memory, making it more suitable for large search spaces where A* would require too much memory to store all nodes in the open and closed lists.
- IDA* combines the space efficiency of Iterative Deepening Depth-First Search (IDDFS) with the optimality of A*. Instead of maintaining a priority queue like A*, IDA* performs a series of depth-limited searches, where the depth limit is determined by a threshold that is increased iteratively. Each iteration explores paths that do not exceed the current threshold, and the threshold is based on the f-cost (sum of g-cost and h-cost).

Iterative Deepening A* algorithm (IDA*) – Artificial intelligence

- The evaluation function in IDA* looks like this:

$$\Rightarrow f(n) = g(n) + h(n)$$
$$f(n) = Actual\ cost + Estimated\ cost$$

- Where h is admissible.
- here,
- f(n) = Total cost evaluation function.
- g(n) = The actual cost from the initial node to the current node.
- h(n) = Heuristic estimated cost from the current node to the goal state. it is based on the approximation according to the problem characteristics.
- Threshold: In each iteration, IDA* sets a threshold based on the f-cost. It only explores nodes where f(n)≤f(n)≤ current threshold.

Iterative Deepening A* algorithm (IDA*) – Artificial intelligence

- Algorithm Steps
- Initial Threshold:
- Set the initial threshold to f(start)=g(start)+h(start), where g(start)=0.
- Depth-Limited Search:
- Perform a depth-first search (DFS) where nodes are only expanded if their f-cost does not exceed the current threshold.
- Update Threshold:
- If the goal node is not found in the current iteration, set the threshold to the minimum f-cost of any node that exceeded the current threshold in the previous iteration.
- Repeat: Repeat the DFS with the new threshold until the goal node is found or no nodes remain to explore

Iterative Deepening A* algorithm (IDA*) – The complete Algorithm

- Step 1: Initialization
- Set the root node as the current node, and find the f-score.
- Sep 2: Set threshold
- Set the cost limit as a threshold for a node i.e the maximum f-score allowed for that node for further explorations.
- Step 3: Node Expansion
- Expand the current node to its children and find f-scores.
- Step 4: Pruning
- If for any node the f-score > threshold, prune that node because it's considered too expensive for that node. and store it in the visited node list.
- Step 5: Return Path
- If the Goal node is found then return the path from the start node Goal node.

Iterative Deepening A* algorithm (IDA*) – The complete Algorithm

- Step 6: Update the Threshold
- If the Goal node is not found then repeat from step 2 by changing the threshold with the minimum pruned value from the visited node list. And Continue it until you reach the goal node.

⇒ (IDA* is more Complex than A*)

IDA* vs A*

- 1. Algorithm Type:
- A*: A* is a best-first search algorithm that explores nodes in the order of their estimated total path cost f(n)=g(n)+h(n), where g(n) is the cost to reach the node and h(n) is the heuristic estimate of the cost from the node to the goal.
- IDA*: IDA* is an iterative deepening version of A*, which uses depth-first search but applies a threshold based on f(n) to limit the depth of the search in each iteration. It increases the threshold progressively until the goal is found.

IDA* vs A*

- 2. Memory Usage:
- A*: Requires significant memory, as it stores all the nodes in the open and closed lists. In the worst case, this can lead to memory consumption growing exponentially with the depth of the search space.
- IDA*: Requires much less memory, similar to depth-first search. It only needs to store the current path and a small amount of additional information, making it much more memory-efficient.

IDA* vs A*

- 3. Time Complexity:
- A*: The time complexity is $O(b^d)$, where bb is the branching factor and dd is the depth of the optimal solution. It explores all possible paths but typically does so faster than uninformed search algorithms.
- IDA*: The time complexity is also $O(b^d)$, but due to its iterative nature, it may revisit nodes multiple times, making it potentially slower in practice compared to A*.

IDA* vs A*

- 4. Space Complexity:

- A*: Space complexity is $O(b^d)$, as it needs to store all explored nodes. This can be a significant limitation for large problems.
- IDA*: Space complexity is $O(b \times d)$, as it only stores the path being explored and the next threshold. This makes IDA* feasible for much larger problems than A*.

IDA* vs A*

- A* is faster in practice and is preferred when memory is sufficient.
- IDA* is more memory-efficient and suitable for large problems but may be slower due to its iterative nature.

⇒ (IDDFS Vs IDA*)

Informed
- → GBFS
- → A*
- → AO*
- → IDA*
- → BFHS
- → HCS

} Read H.W.

BFHS

Recall BFS → Queue

the Process is Same
Simply we use priority Queue
→ the priority is given to the nodes according to Heuristic

Breadth-First Heuristic Search

- Breadth-First Heuristic Search (BFHS) is a search algorithm that combines the principles of Breadth-First Search (BFS) with heuristic guidance to explore the most promising paths first. While traditional BFS explores all nodes at the current depth level before moving on to the next, BFHS introduces a heuristic to prioritize the exploration of nodes that are expected to be closer to the goal.
- Breadth-First Search (BFS): BFS is a graph traversal algorithm that explores nodes level by level. It starts from the root node (or starting node) and explores all its neighbors before moving to the next level of nodes. BFS is guaranteed to find the shortest path in an unweighted graph.
- Heuristic: A heuristic is an estimate of the cost to reach the goal from a given node. In search algorithms, heuristics guide the search process toward the goal more efficiently by prioritizing nodes that appear to be closer to the goal.

Breadth-First Heuristic Search

How BFHS Works

- Initialization: BFHS begins at the start node and uses a queue to manage the nodes that need to be explored. Each node in the queue is associated with a heuristic value, which represents the estimated cost to reach the goal from that node.
- Heuristic-Based Expansion: Instead of exploring all nodes at the current level as in BFS, BFHS prioritizes the expansion of nodes based on their heuristic values. Nodes with lower heuristic values (indicating they are closer to the goal) are explored first.
- Queue Management: Nodes are added to the queue based on their heuristic values. The queue is typically implemented as a priority queue (or min-heap) where the node with the lowest heuristic value is dequeued first.

Steps in BFHS ⇒.

Step 1: Start with Root node | start node, Each node will expand into it's neighbour.

Step 2: When any node is Expanded, the child node are added into Queue, This is a priority Queue i.e nodes with Smallest heuristic value are expanded first.

Step 3: Keep repeating step 2 until goal node is Reached

Breadth-First Heuristic Search

- How BFHS Works
- Goal Test: The algorithm checks if the current node is the goal node. If the goal node is found, the search terminates.
- Path Reconstruction: Once the goal is reached, the path from the start node to the goal can be reconstructed by backtracking through the parent nodes.

- BFS → Complete + optimal for non weighted graph
- DFS → not Complete + not optimal
- UCS → Complete + optimal in weighted graph
- IDDFS → Complete + Complete
- DLS → not Complete + not optimal

- GBFS → not Complete + not optimal
- A* → Complete + optimal
- AO* → Complete + optimal
- IDA* → Complete + optimal
- BFHS → Complete + non optimal

BFS or BFHS

fast => ??

faster.

*Becoz of heuristic all node are not to be exponed

Breadth-First Heuristic Search

- Key Characteristics
- Exploration Order: Unlike pure BFS, which explores all nodes at a given level before moving deeper, BFHS explores nodes based on their heuristic value, leading to a more informed and potentially faster search.
- Optimality: BFHS can be optimal if the heuristic used is admissible (never overestimates the cost to reach the goal) and consistent (satisfies the triangle inequality). In such cases, BFHS is guaranteed to find the shortest path.
- Completeness: BFHS is complete, meaning it will find a solution if one exists, provided that the heuristic does not cause the algorithm to ignore possible paths.

TC ⇒ (Worst Case) ⇒ $O(b^d)$

SC ⇒ (BFHS = BFS) ⇒ $O(b^d)$

BFHS

advantage

* Complete, optimal if heuristics are consistent
* Faster than BFS

disadvantage

- Space complexity is high need more memory.
- Poor heuristic may lead to Poor results.

Breadth-First Heuristic Search

- Advantages and Limitations
- Advantages: → efficiency, speed more than BFS.

Efficiency: By using a heuristic, BFHS can significantly reduce the number of nodes explored compared to BFS, especially in large search spaces.

Informed Search: The heuristic provides guidance, leading to faster convergence to the goal.

Limitations:

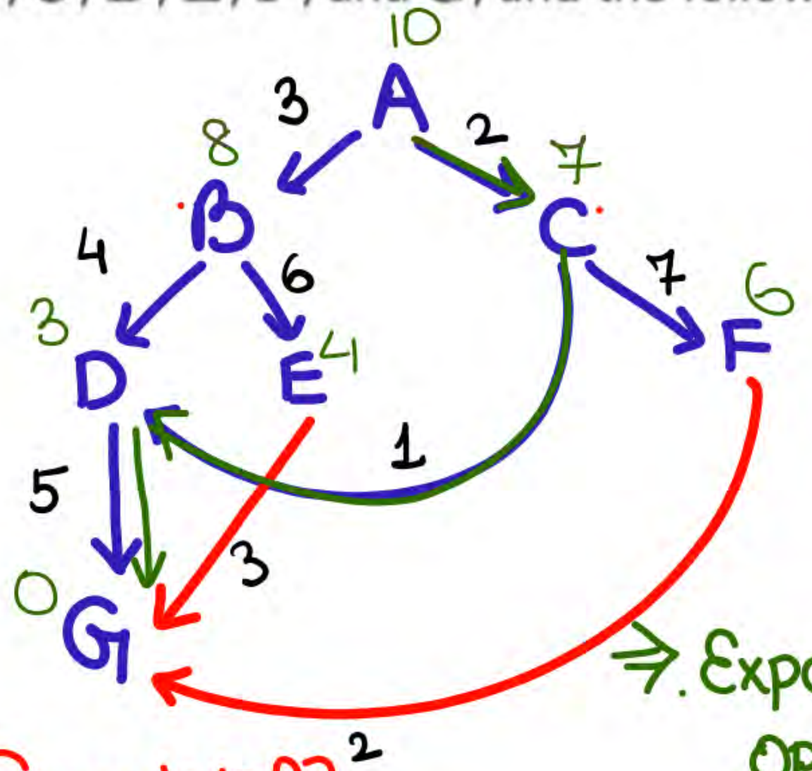
Heuristic Quality: The performance of BFHS heavily depends on the quality of the heuristic. A poor heuristic can lead to inefficient search or even incorrect results if it is not admissible.

Memory Usage: Like BFS, BFHS can require significant memory to store all the nodes at the current depth level.

Problem Statement:

Given a graph with nodes A, B, C, D, E, F, and G, and the following edges with costs:

- $A \to B$ (Cost 3)
- $A \to C$ (Cost 2)
- $B \to D$ (Cost 4)
- $B \to E$ (Cost 6)
- $C \to D$ (Cost 1)
- $C \to F$ (Cost 7)
- $D \to G$ (Cost 5)
- $E \to G$ (Cost 3)
- $F \to G$ (Cost 2)



(as in BFS any node expand ⇒ open list)

⇒ Expanded A, C, D, G (Cost = 8)

Open list ~~C~~ B, D, F
7 8 3 6

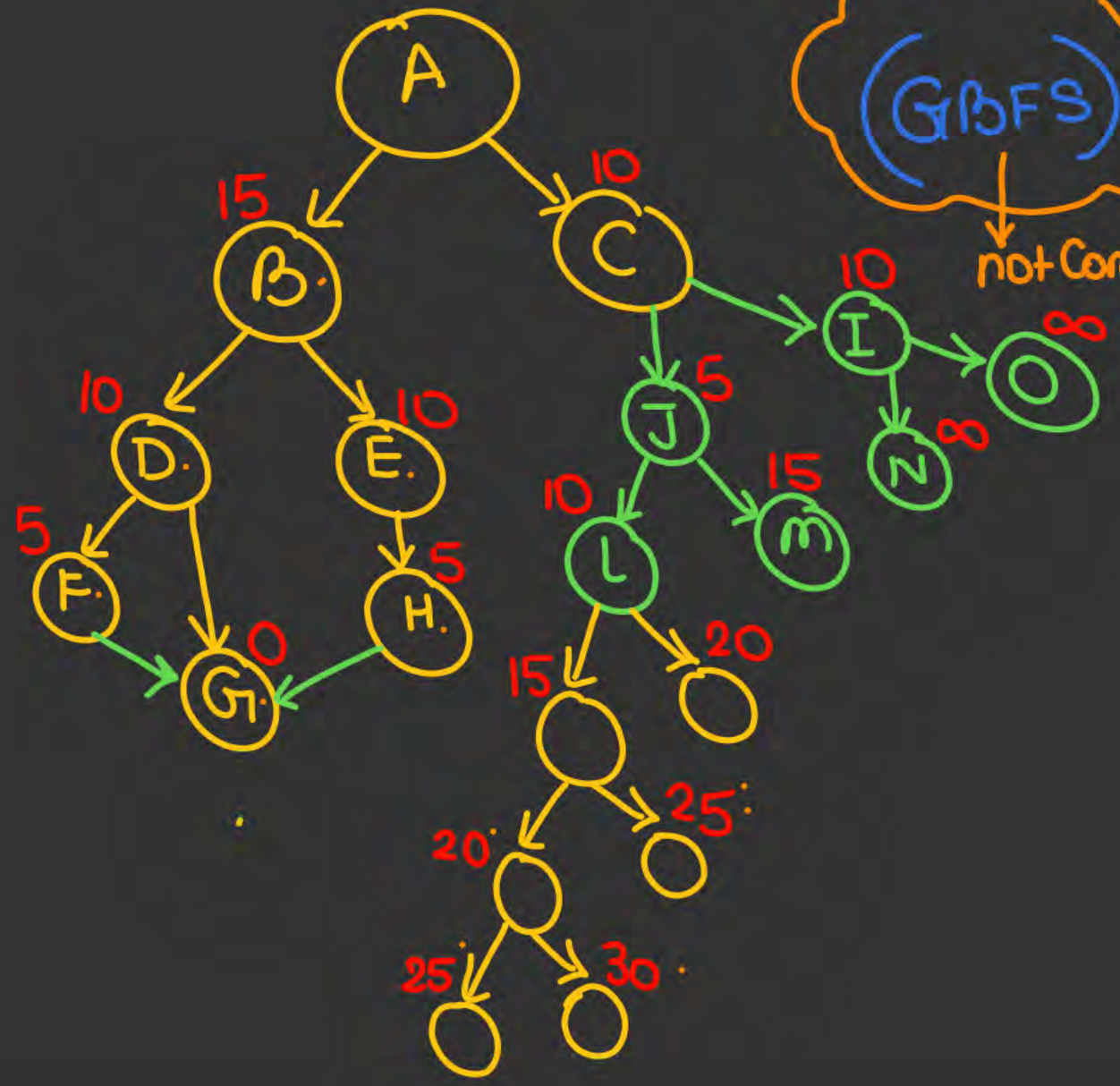
⇒ ~~D~~ F, B, ~~G~~
6 8 0

* Complete?? ⇒ Yes

* optimality ⇒ depend on heuristics.

Find the shortest path from A to G using Breadth-First Heuristic Search, given the following heuristic values:

Node	Heuristic Value (h(n))
A	10
B	8
C	7
D	3
E	4
F	6
G	0



(GBFS) VS (BFHS) → Complete but not optimal

GBFS → not Complete, not optimal

Expanded A, C, J

open list ~~C~~ B, J, I

15 5 10

B, I, L, M

15 10 10 15

h(n) is used.

f(n) is used

BFHS Vs A*

BFHS vs A*

- BFHS is a hybrid algorithm that combines aspects of Breadth-First Search (BFS) and heuristic guidance. It is designed to explore the search space systematically while also using heuristics to guide the search towards the goal.
- A* is a widely used algorithm for finding the shortest path between two points. It uses both the actual cost to reach a node and a heuristic estimate of the cost to reach the goal from that node.

BFHS vs A*

- Optimality: BFHS is not guaranteed to find the optimal path, especially if the heuristic is not well-designed. It might explore more nodes at a certain depth level before considering potentially better nodes at the next level.
- Optimality: A* is guaranteed to find the optimal path if the heuristic is admissible (never overestimates the cost) and consistent.

GBFS (not Complete)

BFHS (Complete)

BFHS vs GBFS*

- BFHS is a hybrid algorithm that combines aspects of Breadth-First Search (BFS) and heuristic guidance. It systematically explores the search space level by level, but with a preference for nodes with better heuristic values.
- GBFS is an algorithm that always expands the node that appears to be closest to the goal according to a heuristic. It focuses purely on the heuristic value (h(n)), without considering the cost to reach the node (g(n)).
- BFHS is generally less efficient than GBFS because it explores all nodes at a particular depth before moving on, potentially expanding unnecessary nodes.

* BFHS need more memory.
* GBFS need less memory.

• BFHS expand more nodes than GBFS.

BFHS vs GBFS*

- Breadth-First Heuristic Search (BFHS):
- Optimality: BFHS is not guaranteed to find the optimal path, especially if the heuristic is not well-crafted. Since it explores all nodes at a given depth level, it might delay finding the optimal path.
- Completeness: BFHS is complete, meaning it will find a solution if one exists, but the solution may not be the optimal one.
- Greedy Best-First Search (GBFS):
- Optimality: GBFS is not optimal. It might find a suboptimal path because it does not consider the actual cost to reach nodes, only the heuristic estimate to the goal.
- Completeness: GBFS is not complete in general. It might fail to find a solution if it gets stuck in a local optimum or if the heuristic leads it down a dead-end path.

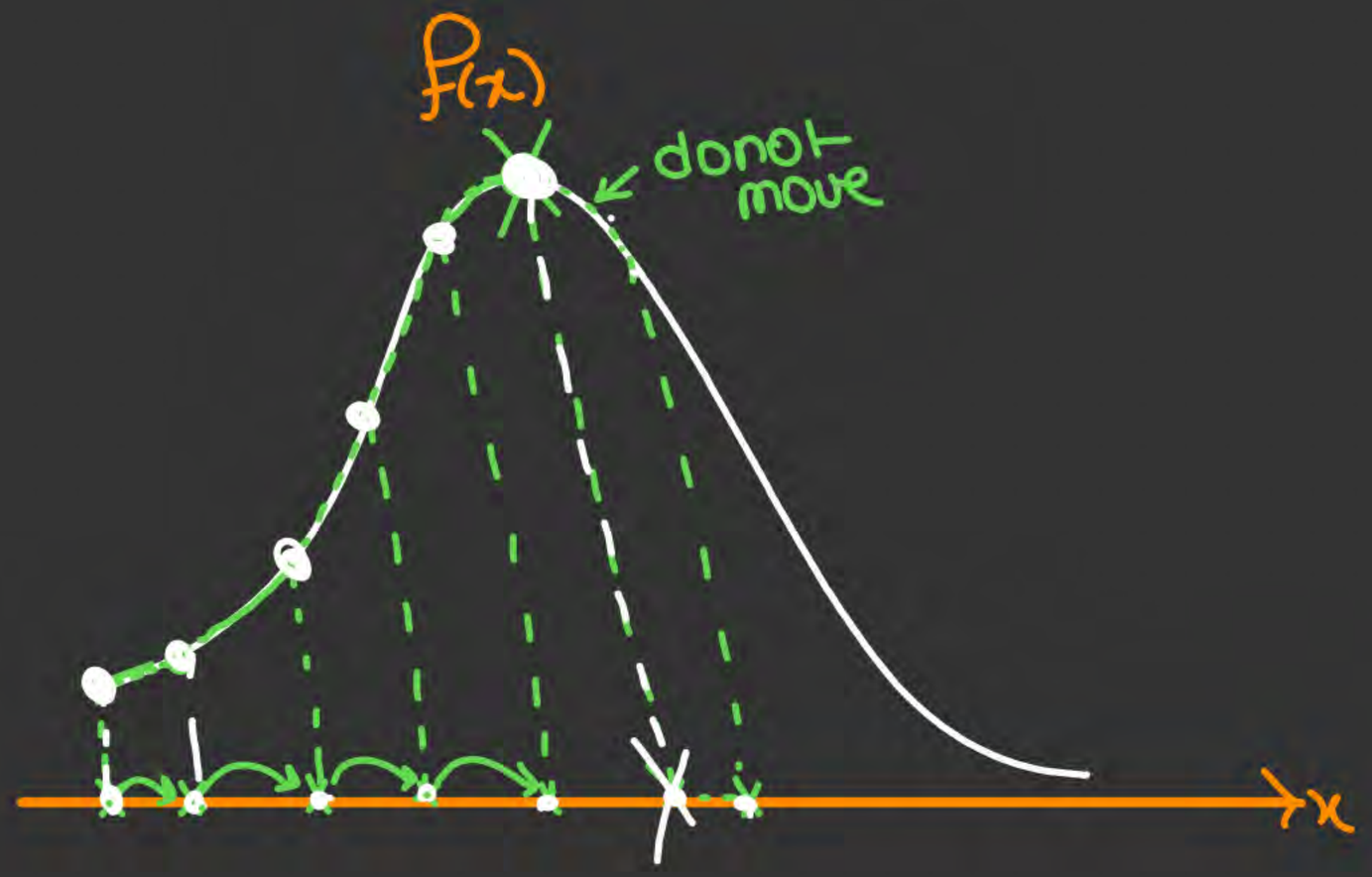
Hill climbing Search ⇒ Local Search algorithm

→ algo stop @ K

→ neither Complete

→ nor optimal

Hill Climbing vs GBFS


f(x)
donot move
x

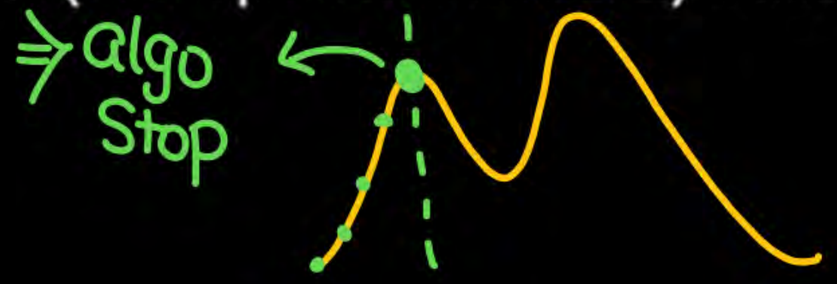
GBFS → local Search
BFHS → global search

Hill Climbing Search

- Hill climbing is a simple optimization algorithm used in Artificial Intelligence (AI) to find the best possible solution for a given problem. It belongs to the family of local search algorithms and is often used in optimization problems where the goal is to find the best solution from a set of possible solutions.
- In Hill Climbing, the algorithm starts with an initial solution and then iteratively makes small changes to it in order to improve the solution. These changes are based on a heuristic function that evaluates the quality of the solution. The algorithm continues to make these small changes until it reaches a local maximum, meaning that no further improvement can be made with the current set of moves.

Hill Climbing Search

- What is Hill Climbing?
- An iterative optimization algorithm.
- It starts with an arbitrary solution and iteratively makes small changes to improve it.
- The algorithm moves towards increasing value (uphill) to find the peak or optimal solution.
- Real-World Analogies
- Like climbing a hill where the peak represents the best solution.
- Can get stuck at local peaks (sub-optimal solutions) instead of the global peak (best possible solution).



Hill Climbing Search

Steps ⇒ 1. Start from Root node

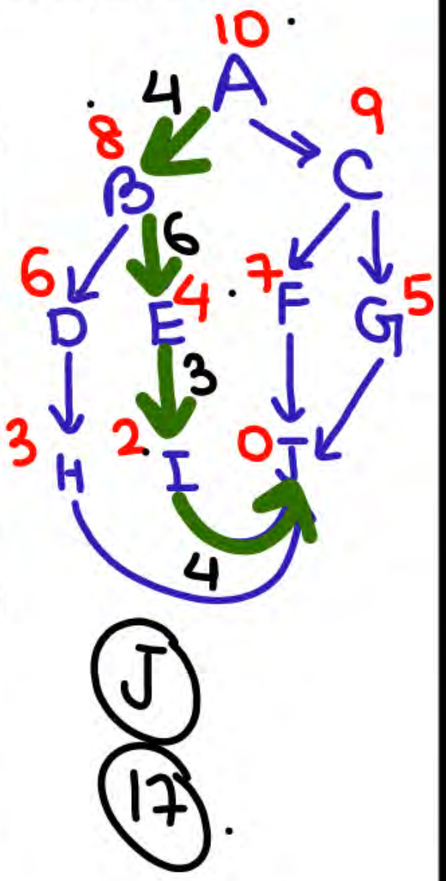
2. move to that neighbour node which has least heuristic value, and heuristic value ≤ heuristic of current node

3. Repeat step 2, stop if goal node reach else stop if all neighbours have heuristics > heuristic of current node

Artificial Intelligence

Consider a graph with the following nodes and edges:

- Nodes: $A, B, C, D, E, F, G, H, I, J$
- Edges with Costs:
  - A to B (Cost 4)
  - A to C (Cost 3)
  - B to D (Cost 5)
  - B to E (Cost 6)
  - C to F (Cost 8)
  - C to G (Cost 7)
  - D to H (Cost 4)
  - E to I (Cost 3)
  - F to J (Cost 2)
  - G to J (Cost 5)
  - H to J (Cost 1)
  - I to J (Cost 4)

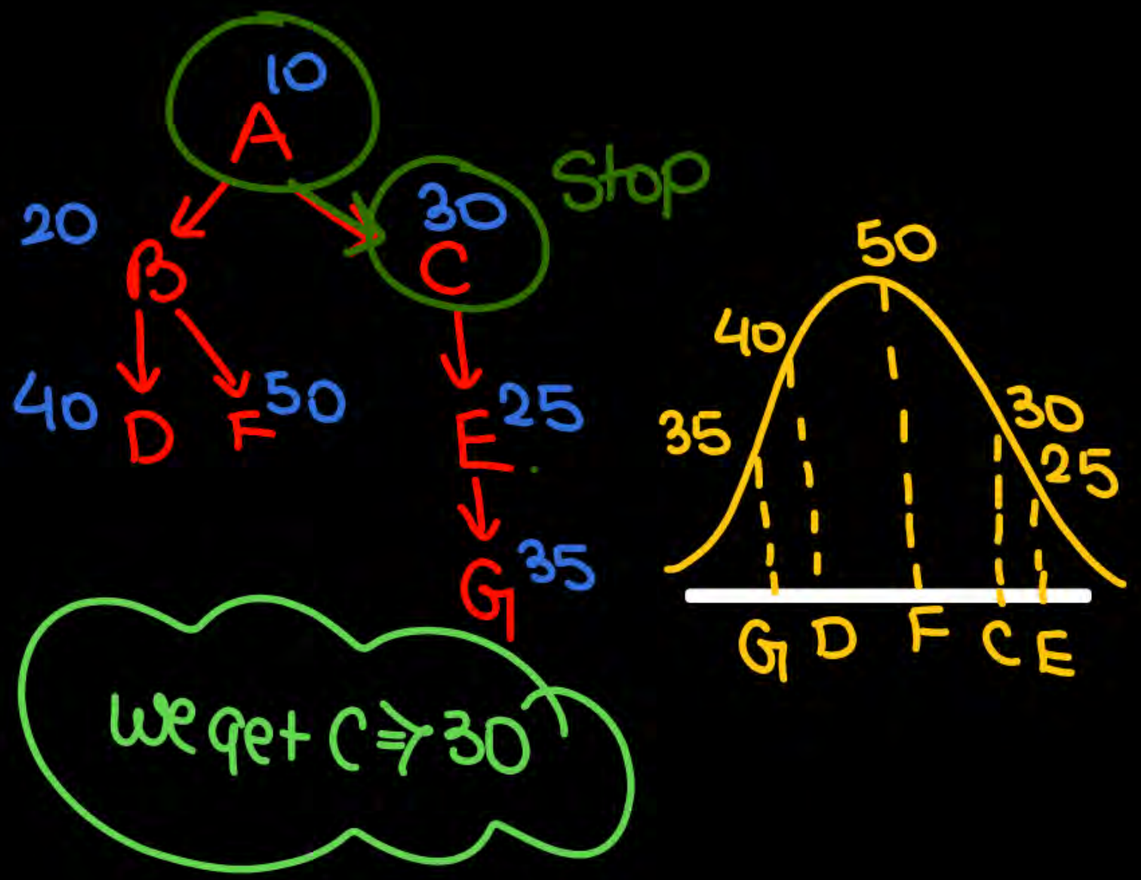


Node	Heuristic Value $h(n)$
A	10
B	8
C	9
D	6
E	4
F	7
G	5
H	3
I	2
J	0

Graph Details:

- Nodes: A, B, C, D, E, F, G
- Values:
  - A (Value 10)
  - B (Value 20)
  - C (Value 30)
  - D (Value 40)
  - E (Value 25)
  - F (Value 50)
  - G (Value 35)

- height values
- not heuristics

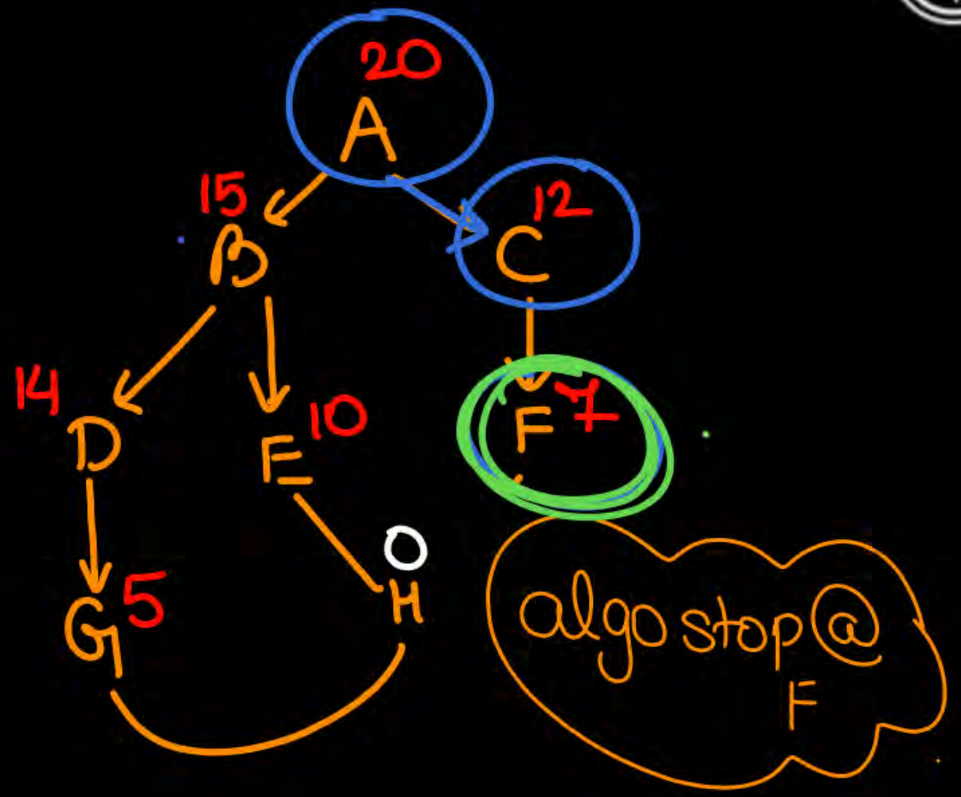


Graph Details:

- Nodes: A, B, C, D, E, F, G, H
- Heuristic Values:
  - A: 20
  - B: 15
  - C: 12
  - D: 14
  - E: 10
  - F: 7
  - G: 5
  - H: 0

Connections:

- A connects to B and C.
- B connects to D and E.
- C connects to F.
- D connects to G.
- E [redacted] connect to H.
- G connects to H.



THANK - YOU